DECISION TREE

Haan, ye **Decision Tree Classification** ka correct program hai. Viva ya practical me agar line-by-line explain karna ho to is tarah bata sakte ho:

---

1. Libraries Import Karna

import pandas as pd

- Pandas library ko import kiya.
- Dataset (CSV file) ko read aur handle karne ke liye use hoti hai.

import matplotlib.pyplot as plt

- Matplotlib visualization ke liye use hoti hai.
- Decision Tree ko graphically display karne ke liye.

from sklearn.tree import DecisionTreeClassifier, plot_tree

- `DecisionTreeClassifier` → Decision Tree model banane ke liye.
- `plot_tree` → Tree ko draw karne ke liye.

from sklearn.preprocessing import LabelEncoder

- Categorical data ko numeric data me convert karne ke liye.

from sklearn.model_selection import train_test_split

- Dataset ko training aur testing data me divide karne ke liye.

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

- Accuracy calculate karne ke liye.
- Confusion Matrix generate karne ke liye.
- Classification Report generate karne ke liye.

---

2. Dataset Load Karna

df = pd.read_csv("Customer_Behaviour.csv")

- CSV file ko read karke DataFrame `df` me store kiya.
- Dataset me customer information hoti hai.

Example:

Gender	Age	EstimatedSalary	Purchased
Male	35	50000	1
Female	28	30000	0

---

3. Gender Column Encode Karna

le = LabelEncoder()

- Label Encoder object create kiya.

df["Gender"] = le.fit_transform(df["Gender"])

- Gender values ko numbers me convert kiya.

Example:

Gender
Male
Female

Convert hoga:

Gender
1
0

Machine Learning algorithms text directly process nahi kar sakte, isliye encoding zaruri hai.

---

4. Features Aur Target Define Karna

X = df[["Gender", "Age", "EstimatedSalary"]]

- Input features select kiye.
- Model in values ke basis par prediction karega.

y = df["Purchased"]

- Target column select kiya.
- Ye batata hai customer ne product purchase kiya ya nahi.

---

5. Dataset Split Karna

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

Parameters:

- `X` → Features
- `y` → Target
- `test_size=0.3`
  - 30% data testing ke liye
  - 70% data training ke liye
- `random_state=42`
  - Har baar same split milega.

Example:

100 records

- 70 → Training
- 30 → Testing

---

6. Decision Tree Model Banana

model = DecisionTreeClassifier(criterion="entropy")

Decision Tree classifier create kiya.

Entropy

Decision Tree node split karne ke liye Information Gain use karta hai.

Entropy impurity measure hai.

Jitni entropy kam hogi, data utna pure hoga.

---

7. Model Train Karna

model.fit(X_train, y_train)

- Model ko training data diya.
- Decision Tree learn karta hai ki purchase hoga ya nahi.

---

8. Prediction Karna

y_pred = model.predict(X_test)

- Testing data par prediction kiya.
- Result `y_pred` me store hua.

Example:

Actual:

[1,0,1,1]

Predicted:

[1,0,0,1]

---

9. Accuracy Calculate Karna

print("Accuracy:", accuracy_score(y_test, y_pred))

Accuracy formula:

Accuracy=\frac{Correct\ Predictions}{Total\ Predictions}

Example:

90 correct predictions out of 100

Accuracy = 90%

Output:

Accuracy: 0.90

---

10. Classification Report

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Ye metrics deta hai:

Precision

Kitni positive predictions actually correct thi.

Recall

Actual positives me se kitni identify hui.

F1-Score

Precision aur Recall ka balance.

Example Output:

precision recall f1-score support

0 0.90 0.88 0.89 50
1 0.87 0.91 0.89 50

---

11. Confusion Matrix

```
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Example:

```
[[45 5]
 [3 47]]
```

Meaning:

	Predicted No	Predicted Yes
Actual No	45	5
Actual Yes	3	47

- 45 = True Negative
- 47 = True Positive
- 5 = False Positive
- 3 = False Negative

---

12. Figure Size Set Karna

```
plt.figure(figsize=(12, 8))
```

- Tree diagram ka size set kiya.
- Width = 12
- Height = 8

---

13. Decision Tree Plot Karna

```
plot_tree(
    model,
    feature_names=["Gender", "Age", "EstimatedSalary"],
    class_names=["Not Purchased", "Purchased"],
    filled=True
```

)

Parameters:

model

- Trained Decision Tree model.

feature_names

- Tree me feature names show honge.

class_names

- Output classes ke names.

filled=True

- Nodes ko colors se fill karega.
- Visualization easy ho jayegi.

---

14. Tree Display Karna

plt.show()

- Decision Tree graph screen par display karega.

Viva Me 1 Minute Explanation

"Sabse pehle maine required libraries import ki. Phir CSV dataset load kiya aur Gender column ko LabelEncoder ki help se numeric values me convert kiya. Uske baad Gender, Age aur EstimatedSalary ko features aur Purchased ko target variable banaya. Dataset ko 70% training aur 30% testing data me split kiya. DecisionTreeClassifier with entropy criterion create karke model train kiya. Testing data par prediction ki aur accuracy, classification report aur confusion matrix calculate ki. Finally, plot_tree function ki help se Decision Tree visualize ki."

Tumhare output ka matlab ye hai:

1. Accuracy

Accuracy: 0.85

Iska matlab:

Accuracy=\frac{102}{120}=0.85=85%

- Total test records = 120
- Correct predictions = 102
- Accuracy = 85%

Interpretation: Model ne 120 me se 102 customers ko sahi predict kiya.

---

2. Classification Report

Class 0 (Not Purchased)

Precision = 0.89
Recall = 0.86
F1-score = 0.88
Support = 73

- 73 customers ne purchase nahi kiya.
- Jab model ne "Not Purchased" predict kiya, to 89% cases me sahi tha.
- Actual Not Purchased customers me se 86% ko correctly identify kiya.

Class 1 (Purchased)

Precision = 0.80
Recall = 0.83
F1-score = 0.81
Support = 47

- 47 customers ne purchase kiya.
- Jab model ne "Purchased" predict kiya, to 80% cases me sahi tha.
- Actual Purchased customers me se 83% ko correctly identify kiya.

---

3. Confusion Matrix

[[63 10]
 [8 39]]

Matrix ko aise padho:

Actual / Predicted	Not Purchased (0)	Purchased (1)
Not Purchased (0)	63	10
Purchased (1)	8	39

Meaning

True Negative (63)
63

- Actual = Not Purchased
- Predicted = Not Purchased

Model ne sahi prediction kiya.

False Positive (10)
10

- Actual = Not Purchased
- Predicted = Purchased

Model ne galat purchase predict kiya.

False Negative (8)
8

- Actual = Purchased
- Predicted = Not Purchased

Model customer purchase karega, lekin model ne nahi karega predict kiya.

True Positive (39)
39

- Actual = Purchased
- Predicted = Purchased

Model ne sahi purchase predict kiya.

Viva Explanation

Agar examiner pooche:

"Output explain karo."

To bolo:

> "Model ki accuracy 85% aayi hai, matlab 120 testing records me se 102 predictions correct hain. Confusion matrix ke according 63 customers ko correctly Not Purchased aur 39 customers ko correctly Purchased classify kiya gaya. 10 customers ko galti se Purchased aur 8 customers ko galti se Not Purchased classify kiya gaya. Classification report me Purchased class ke liye precision 80%, recall 83% aur F1-score 81% hai, jo model ki achhi performance show karta hai."

Ye explanation viva ke liye bilkul sufficient hai. 👍

2. NAVEBAYES

Ye code **StudentsPerformance.csv** dataset par **Gaussian Naive Bayes Classification** perform karta hai. Tumhara output bhi bahut achha hai (**97.5% accuracy**).

---

1. Libraries Import Karna

import pandas as pd

- Pandas library import ki.

- CSV file read aur data handle karne ke liye.

import seaborn as sns

- Seaborn library import ki.
- Confusion Matrix ka heatmap banane ke liye.

import matplotlib.pyplot as plt

- Graph aur charts display karne ke liye.

---

2. Machine Learning Libraries

from sklearn.preprocessing import LabelEncoder

- Text data ko numeric form me convert karne ke liye.

from sklearn.model_selection import train_test_split

- Dataset ko training aur testing data me divide karne ke liye.

from sklearn.naive_bayes import GaussianNB

- Gaussian Naive Bayes classifier import kiya.

from sklearn.metrics import accuracy_score, confusion_matrix

- Accuracy aur Confusion Matrix calculate karne ke liye.

---

3. Dataset Load Karna

df = pd.read_csv("StudentsPerformance.csv")

- CSV file read ki.
- DataFrame `df` me store hua.

---

4. Target Column Banana

df["Result"] = (df["math score"] >= 40).astype(int)

Math score ke basis par Pass/Fail banaya.

Example

math score	Result
75	1
90	1
25	0
35	0

Yahan:

1 = Pass
0 = Fail

---

5. Label Encoder Object Banana

le = LabelEncoder()

LabelEncoder object create kiya.

---

6. Categorical Data Encode Karna

Gender

df["gender"] = le.fit_transform(df["gender"])

Example:

gender
female
male

Convert hoga:

gender
0
1

Race/Ethnicity

df["race/ethnicity"] = le.fit_transform(df["race/ethnicity"])

Example:

group A = 0
group B = 1
group C = 2
group D = 3
group E = 4

Parent Education

df["parental level of education"] = le.fit_transform(df["parental level of education"])

Education categories numbers me convert ho jaati hain.

Lunch

df["lunch"] = le.fit_transform(df["lunch"])

Example:

free/reduced = 0
standard = 1

Test Preparation Course

df["test preparation course"] = le.fit_transform(df["test preparation course"])

Example:

none = 0
completed = 1

---

7. Features Aur Target Define Karna

X = df.drop("Result", axis=1)

Features:

- gender
- race/ethnicity
- parental education
- lunch
- test preparation course
- math score
- reading score
- writing score

---

y = df["Result"]

Target variable:

Pass = 1
Fail = 0

---

8. Dataset Split Karna

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Meaning

- 80% data training ke liye
- 20% data testing ke liye

Agar 1000 records hain:

800 → Training
200 → Testing

---

9. Naive Bayes Model Banana

```
nb = GaussianNB()
```

Gaussian Naive Bayes classifier create kiya.

Gaussian Naive Bayes continuous numerical data ke liye use hota hai.

---

10. Model Train Karna

```
nb.fit(X_train, y_train)
```

Model training data se patterns learn karta hai.

---

11. Prediction Karna

```
y_pred = nb.predict(X_test)
```

Testing data par prediction karta hai.

Example:

Actual : [1,1,0,1]
Predicted : [1,1,0,0]

---

12. Accuracy Calculate Karna

print("Naive Bayes Accuracy:", accuracy_score(y_test, y_pred))

Tumhara output:

Naive Bayes Accuracy: 0.975

Matlab:

Accuracy=97.5%

Model ne 97.5% predictions sahi kiye.

Ye bahut achhi accuracy hai.

---

13. Confusion Matrix

cm = confusion_matrix(y_test, y_pred)

Tumhara output:

[[11 0]
[5 184]]

Table form me:

Actual / Predicted	Fail (0)	Pass (1)
Fail (0)	11	0
Pass (1)	5	184

Meaning

True Negative = 11

11

Actual Fail tha aur model ne bhi Fail predict kiya.

Correct Prediction ✅

False Positive = 0

0

Actual Fail tha lekin Pass predict nahi hua.

Perfect Result ✅

False Negative = 5

5

Actual Pass tha lekin model ne Fail predict kiya.

Galat Prediction ❌

True Positive = 184

184

Actual Pass tha aur model ne bhi Pass predict kiya.

Correct Prediction ✅

---

14. Heatmap Banana

plt.figure(figsize=(6,4))

Graph size set karta hai.

---

sns.heatmap(cm, annot=True, fmt='d')

Confusion Matrix ko heatmap form me show karta hai.

annot=True

Numbers boxes ke andar show honge.

fmt='d'

Values integer format me show hongi.

---

plt.title("Naive Bayes Confusion Matrix")

Graph title set karta hai.

---

plt.xlabel("Predicted")

X-axis = Predicted values

---

plt.ylabel("Actual")

Y-axis = Actual values

---

plt.show()

Heatmap display karta hai.

Viva Explanation (Short)

"Maine StudentsPerformance dataset load kiya aur math score ke basis par Pass/Fail target column create kiya. LabelEncoder ki madad se categorical data ko numeric form me convert kiya. Dataset ko 80% training aur 20% testing data me split kiya. Gaussian Naive Bayes classifier train kiya aur testing data par prediction ki. Model ki accuracy 97.5% aayi. Confusion Matrix ke according 11 fail students aur 184 pass students correctly classify hue, jabki sirf 5 pass students ko galti se fail predict kiya gaya."

3. SVM

Haan, chalo is **SVM (Support Vector Machine)** code ko line-by-line samajhte hain.

---

1. Libraries Import Karna

import pandas as pd

- Pandas library import ki.
- CSV file read aur data handle karne ke liye use hoti hai.

---

import seaborn as sns

- Seaborn library import ki.
- Confusion Matrix ka heatmap banane ke liye.

---

import matplotlib.pyplot as plt

- Graph aur charts display karne ke liye.

---

2. Machine Learning Libraries

from sklearn.preprocessing import LabelEncoder

- Text data ko numeric values me convert karne ke liye.

Example:

rain → 1
no rain → 0

---

from sklearn.model_selection import train_test_split

- Dataset ko training aur testing parts me divide karne ke liye.

---

from sklearn.svm import SVC

- SVM (Support Vector Machine) classifier import kiya.

---

from sklearn.metrics import accuracy_score, confusion_matrix

- Accuracy calculate karne ke liye.
- Confusion Matrix generate karne ke liye.

---

3. Dataset Load Karna

df = pd.read_csv("weather_forecast_data.csv")

- CSV file read ki.
- DataFrame `df` me store hua.

Dataset ke columns:

Temperature
Humidity
Wind_Speed
Cloud_Cover
Pressure
Rain

---

4. Rain Column Encode Karna

le = LabelEncoder()

- LabelEncoder object create kiya.

---

df["Rain"] = le.fit_transform(df["Rain"])

Rain column ko numbers me convert kiya.

Example:

Rain
rain
no rain

Convert hoga:

Rain
1
0

Machine Learning algorithms text directly process nahi kar sakte, isliye encoding zaruri hai.

---

5. Features Select Karna

```
X = df[["Temperature", "Humidity", "Wind_Speed",
        "Cloud_Cover", "Pressure"]]
```

Features (inputs) select kiye.

Ye values model ko di jayengi.

Feature
Temperature
Humidity
Wind_Speed
Cloud_Cover
Pressure

---

6. Target Variable Select Karna

y = df["Rain"]

Target variable:

0 = no rain
1 = rain

Model isi ko predict karega.

---

7. Dataset Split Karna

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Meaning

- X = Features
- y = Target

test_size = 0.2

- 20% Testing Data
- 80% Training Data

Agar dataset me 1000 rows hain:

800 → Training
200 → Testing

---

random_state = 42

Har baar same train-test split milega.

---

8. SVM Model Banana

svm = SVC(kernel="linear")

SVM classifier create kiya.

Linear Kernel

- Data ko ek straight line (2D) ya hyperplane (multi-dimensional) se separate karta hai.

SVM ka objective:

> Dono classes ke beech maximum margin wala boundary find karna.

---

9. Model Train Karna

svm.fit(X_train, y_train)

Model training data se learn karta hai.

Yahan SVM ye seekhta hai:

Temperature
Humidity
Wind Speed
Cloud Cover
Pressure

ke basis par rain hogi ya nahi.

---

10. Prediction Karna

y_pred = svm.predict(X_test)

Testing data par prediction karta hai.

Example:

Actual:

[1, 0, 1, 1]

Predicted:

[1, 0, 0, 1]

---

11. Accuracy Calculate Karna

print("SVM Accuracy:", accuracy_score(y_test, y_pred))

Accuracy formula:

Accuracy=\frac{Correct\ Predictions}{Total\ Predictions}

Example:

SVM Accuracy: 0.92

Matlab:

92% predictions correct hain

---

12. Confusion Matrix Banana

```
cm = confusion_matrix(y_test, y_pred)
```

Confusion Matrix create hoti hai.

Example:

```
[[80 10]
 [ 5 105]]
```

```
print("\nConfusion Matrix:")
print(cm)
```

Matrix screen par print hogi.

Confusion Matrix Meaning

Actual / Predicted	No Rain	Rain
No Rain	TN	FP
Rain	FN	TP

TN (True Negative)

Correctly predicted no rain.

TP (True Positive)

Correctly predicted rain.

FP (False Positive)

No rain tha, lekin rain predict kar diya.

FN (False Negative)

Rain thi, lekin no rain predict kar diya.

---

13. Figure Size Set Karna

plt.figure(figsize=(6,4))

Heatmap ka size set karta hai.

- Width = 6
- Height = 4

---

14. Heatmap Banana

sns.heatmap(cm, annot=True, fmt='d')

Confusion Matrix ko graphical form me show karta hai.

annot=True

Har box ke andar number dikhata hai.

fmt='d'

Numbers integer form me show honge.

---

15. Graph Title

plt.title("SVM Confusion Matrix")

Heatmap ka title set karta hai.

---

16. X-axis Label

plt.xlabel("Predicted")

X-axis = Predicted values

---

17. Y-axis Label

plt.ylabel("Actual")

Y-axis = Actual values

---

18. Heatmap Display Karna

plt.show()

Confusion Matrix heatmap screen par display karta hai.

Viva Me Short Explanation

> "Maine weather_forecast_data.csv dataset load kiya aur Rain column ko LabelEncoder ki help se numeric values me convert kiya. Temperature, Humidity, Wind Speed, Cloud Cover aur Pressure ko features aur Rain ko target variable banaya. Dataset ko 80% training aur 20% testing data me divide kiya. Linear kernel ke saath SVM model train kiya, prediction ki aur accuracy calculate ki. Confusion Matrix generate karke Seaborn heatmap ki madad se visualize kiya."

Viva Explanation

Dataset

- Weather Forecast Dataset
- Total Features = 5
    - Temperature
    - Humidity
    - Wind_Speed
    - Cloud_Cover
    - Pressure

Target Variable

Rain

Values:

rain

no rain

LabelEncoder ke baad:

rain = 1
no rain = 0

(ya reverse bhi ho sakta hai, encoder par depend karta hai)

Features and Target

```
X = df[["Temperature", "Humidity",
       "Wind_Speed", "Cloud_Cover", "Pressure"]]

y = df["Rain"]
```

- X = Input Features
- y = Output Variable

Train-Test Split

test_size=0.2

- 80% Training Data
- 20% Testing Data

SVM Model

svm = SVC(kernel="linear")

- SVC = Support Vector Classifier
- Linear Kernel use kiya hai
- Data ko classes me separate karta hai

Training

svm.fit(X_train, y_train)

Model training data se learn karta hai.

Prediction

y_pred = svm.predict(X_test)

Testing data par prediction karta hai.

Accuracy

accuracy_score(y_test, y_pred)

Model kitne predictions sahi karta hai uska percentage.

Confusion Matrix

confusion_matrix(y_test, y_pred)

Batati hai:

- True Positive
- True Negative
- False Positive
- False Negative

LOGISTIC REGRESSION

Haan, ye **Logistic Regression** ka correct program hai. Chalo isse **line-by-line** samajhte hain aur tumhare output ko bhi explain karte hain.

---

1. Libraries Import Karna

import pandas as pd

- Pandas library import ki.
- CSV file read aur data handle karne ke liye.

---

import matplotlib.pyplot as plt

- Graph aur charts display karne ke liye.

---

import seaborn as sns

- Confusion Matrix ko heatmap form me dikhane ke liye.

---

2. Machine Learning Libraries

from sklearn.preprocessing import LabelEncoder, StandardScaler

LabelEncoder

Categorical data ko numeric me convert karta hai.

Example:

Male → 1
Female → 0

StandardScaler

Features ko scale karta hai.

Formula:

genui{"math_block_widget_always_prefetch_v2":{"content":"z=\frac{x-\mu}{\sigma}"}}

Logistic Regression me scaling performance improve karti hai.

---

from sklearn.model_selection import train_test_split

Dataset ko training aur testing me divide karne ke liye.

---

from sklearn.linear_model import LogisticRegression

Logistic Regression classifier import kiya.

---

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

- Accuracy calculate karne ke liye
- Confusion Matrix generate karne ke liye
- Classification Report generate karne ke liye

---

3. Dataset Load Karna

df = pd.read_csv("Customer_Behaviour.csv")

CSV file ko DataFrame me load kiya.

---

print("Dataset Preview:")
print(df.head())

Pehli 5 rows display karta hai.

Output:

User ID Gender Age EstimatedSalary Purchased

---

4. Gender Encode Karna

le = LabelEncoder()

LabelEncoder object create kiya.

---

df["Gender"] = le.fit_transform(df["Gender"])

Gender column ko numeric values me convert kiya.

Example:

Gender
Female
Male

Convert:

Gender
0
1

---

5. Features Select Karna

X = df[["Gender", "Age", "EstimatedSalary"]]

Input Features:

- Gender
- Age
- EstimatedSalary

Ye values model ko di jayengi.

---

6. Target Variable

y = df["Purchased"]

Target column.

Values:

0 = Not Purchased
1 = Purchased

Model isi ko predict karega.

---

7. Dataset Split Karna

```
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.3, random_state=42
)
```

Meaning

70% → Training Data
30% → Testing Data

Agar total records 400 hain:

280 → Training
120 → Testing

---

random_state=42

Har baar same split milega.

---

8. Feature Scaling

```
scaler = StandardScaler()
```

Scaler object create kiya.

```
X_train = scaler.fit_transform(X_train)
```

Training data scale kiya.

```
X_test = scaler.transform(X_test)
```

Testing data ko bhi same scaling se transform kiya.

---

9. Logistic Regression Model Banana

model = LogisticRegression(max_iter=1000)

Model create kiya.

max_iter=1000

Model ko maximum 1000 iterations tak train karne ki permission di.

Kabhi-kabhi default iterations kam pad jaati hain.

---

10. Model Train Karna

model.fit(X_train, y_train)

Model training data se learn karta hai.

Ye seekhta hai:

Gender
Age
EstimatedSalary

ke basis par purchase hoga ya nahi.

---

11. Prediction Karna

y_pred = model.predict(X_test)

Testing data par prediction karta hai.

Example:

Actual : [1,0,1,1]
Predicted : [1,0,0,1]

---

12. Accuracy

print("\nAccuracy:", accuracy_score(y_test, y_pred))

Output:

Accuracy: 0.8583333333333333

Matlab:

Accuracy=85.83%

Model ne lagbhag 86% predictions sahi kiye.

---

13. Classification Report

print(classification_report(y_test, y_pred))

Output:

Class 0:
Precision = 0.83
Recall = 0.97
F1 = 0.89

Class 1:
Precision = 0.94
Recall = 0.68
F1 = 0.79

Class 0 (Not Purchased)

Support = 73

73 customers ne purchase nahi kiya.

Precision = 0.83

Jab model ne "Not Purchased" bola:

83% times correct tha

Recall = 0.97

Actual Not Purchased customers me se:

97% correctly identify kiye

Class 1 (Purchased)
Support = 47

47 customers ne purchase kiya.

Precision = 0.94

Jab model ne "Purchased" bola:

94% times correct tha

Recall = 0.68

Actual purchasers me se:

68% hi correctly identify hue

---

14. Confusion Matrix

Output:

```
[[71  2]
 [15 32]]
```

Table form me:

Actual / Predicted	Not Purchased	Purchased
Not Purchased	71	2
Purchased	15	32

True Negative = 71

71

Actual = Not Purchased

Predicted = Not Purchased

Correct ✅

False Positive = 2

2

Actual = Not Purchased

Predicted = Purchased

Wrong ❌

False Negative = 15

15

Actual = Purchased

Predicted = Not Purchased

Wrong ❌

True Positive = 32

32

Actual = Purchased

Predicted = Purchased

Correct ✅

---

15. Heatmap

plt.figure(figsize=(6,4))

Figure size set karta hai.

```
sns.heatmap(cm,
            annot=True,
            fmt="d",
            cmap="Blues")
```

Confusion Matrix ko graphical form me display karta hai.

annot=True

Cells me numbers show karega.

fmt="d"

Integers display karega.

cmap="Blues"

Blue color theme use karega.

---

plt.title("Logistic Regression Confusion Matrix")

Graph title set karta hai.

---

plt.xlabel("Predicted")

X-axis = Predicted Values

---

plt.ylabel("Actual")

Y-axis = Actual Values

---

plt.show()

Heatmap display karta hai.

Viva Explanation (1 Minute)

> "Maine Customer_Behaviour dataset load kiya aur Gender column ko LabelEncoder se numeric values me convert kiya. Gender, Age aur EstimatedSalary ko features aur Purchased ko target variable banaya. Dataset ko 70% training aur 30% testing data me split kiya. StandardScaler se features scale kiye aur Logistic Regression model train kiya. Testing data par prediction ki aur 85.83% accuracy mili. Confusion Matrix ke according 71 non-purchasing customers aur 32 purchasing customers correctly classify hue, jabki 17 predictions galat hui."

LINEAR REGRESSION

Ye **Linear Regression** ka code hai jo **StudentsPerformance.csv** dataset me **Math Score predict** karta hai.

---

1. Libraries Import Karna

import pandas as pd

- Pandas library import ki.
- CSV file read karne ke liye.

---

import matplotlib.pyplot as plt

- Graph plot karne ke liye.

---

from sklearn.preprocessing import LabelEncoder

- Text data ko numeric values me convert karne ke liye.

---

from sklearn.model_selection import train_test_split

- Dataset ko training aur testing me divide karne ke liye.

---

from sklearn.linear_model import LinearRegression

- Linear Regression algorithm import kiya.

---

from sklearn.metrics import mean_squared_error, r2_score

- Model evaluate karne ke liye.

---

2. Dataset Load Karna

df = pd.read_csv("StudentsPerformance.csv")

Dataset ko DataFrame `df` me load kiya.

---

3. Label Encoder Banana

le = LabelEncoder()

Object create kiya.

---

4. Categorical Columns Encode Karna

df["gender"] = le.fit_transform(df["gender"])

Example:

female → 0
male → 1

---

df["race/ethnicity"] = le.fit_transform(df["race/ethnicity"])

Example:

group A → 0
group B → 1
group C → 2
group D → 3
group E → 4

---

df["parental level of education"] = le.fit_transform(df["parental level of education"])

Education categories ko numbers me convert kiya.

---

df["lunch"] = le.fit_transform(df["lunch"])

Example:

free/reduced → 0
standard → 1

---

df["test preparation course"] = le.fit_transform(df["test preparation course"])

Example:

none → 0
completed → 1

---

5. Features Select Karna

```
X = df[[
    "gender",
    "race/ethnicity",
    "parental level of education",
    "lunch",
    "test preparation course",
    "reading score",
    "writing score"
]]
```

Ye sab input features hain.

Model inke basis par prediction karega.

---

6. Target Variable

```
y = df["math score"]
```

Target variable:

Math Score

Model isi score ko predict karega.

---

7. Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Meaning:

- 80% Training Data
- 20% Testing Data

Agar 1000 records hain:

800 → Training
200 → Testing

---

8. Model Banana

```
model = LinearRegression()
```

Linear Regression model create kiya.

Linear Regression equation:

$$y=b_0+b_1x_1+b_2x_2+\cdots+b_nx_n$$

Yahan:

- y = Math Score
- x = Features

---

9. Model Train Karna

```
model.fit(X_train, y_train)
```

Training data se relationship learn karta hai.

---

10. Prediction Karna

```
y_pred = model.predict(X_test)
```

Testing data ke Math Scores predict karta hai.

Example:

Actual Math Score = 72
Predicted Math Score = 70.5

---

11. R^2 Score

```
print("R2 Score:", r2_score(y_test, y_pred))
```

Tumhara Output:

R2 Score: 0.8838026201112225

Matlab:

88.38%

Interpretation:

> Model Math Score me jo variation hai uska lagbhag **88.38% explain kar pa raha hai**.

Ye bahut achha result hai. ✅

R^2 Value Guide

R^2	Meaning
1.0	Perfect
0.8+	Very Good
0.6+	Good
0.5 se kam	Weak

Tumhara:

0.8838

Very Good Model ✅

---

12. Mean Squared Error (MSE)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))

Tumhara Output:

Mean Squared Error: 28.275284506327317

Formula:

MSE=\frac{1}{n}\sum (y-\hat y)^2

Meaning:

- Prediction error measure karta hai.
- Jitna kam MSE hoga utna better model.

Tumhara:

28.27

Reasonably low error hai. ✅

---

13. Graph Banana

plt.figure(figsize=(8,5))

Graph size set karta hai.

---

plt.scatter(y_test, y_pred)

Scatter Plot create karta hai.

X-axis
Actual Math Score

Y-axis
Predicted Math Score

Har dot ek student ko represent karta hai.

---

plt.xlabel("Actual Math Score")

X-axis ka naam.

---

plt.ylabel("Predicted Math Score")

Y-axis ka naam.

---

plt.title("Linear Regression: Actual vs Predicted")

Graph ka title.

---

plt.show()

Graph display karta hai.

Dot-Dot Wala Graph Kya Dikhata Hai?

Jo dots tumhe dikh rahe hain:

- Har dot = 1 student
- X = Actual Math Score
- Y = Predicted Math Score

Agar dots diagonal line ke paas hon

To prediction achhi hai. ✅

Agar dots bahut scattered hon

To prediction weak hai. ❌

Tumhara R^2 = **0.8838** hai, isliye dots generally ek trend/diagonal ke aas-paas honge, jo good model performance dikhata hai.

Viva Explanation

"Linear Regression model ko StudentsPerformance dataset par train kiya gaya. Gender, race, parental education, lunch, test preparation course, reading score aur writing score ko features aur math score ko target variable banaya gaya. Model ne 88.38% variance explain kiya ($R^2 = 0.8838$) aur Mean Squared Error 28.27 aaya, jo achhi prediction performance show karta hai. Scatter plot me actual aur predicted math scores compare kiye gaye hain."

CLUSTERING 2CODE EASY

Bilkul. Main tumhare **simple K-Means Clustering code** ko line-by-line explain karta hoon.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.cluster import KMeans
```

---

1. Pandas Import

```
import pandas as pd
```

- Pandas library import ki.
- CSV file read karne aur data handle karne ke liye.

---

2. Matplotlib Import

```
import matplotlib.pyplot as plt
```

- Graph plot karne ke liye.

---

3. LabelEncoder Import

```
from sklearn.preprocessing import LabelEncoder
```

- Text data ko numbers me convert karne ke liye.

Example:

Yes → 1

No → 0

---

4. KMeans Import

from sklearn.cluster import KMeans

- K-Means clustering algorithm import kiya.

---

5. Dataset Load

df = pd.read_csv("weather.csv")

CSV file ko DataFrame me load kiya.

Tumhara dataset:

MinTemp	MaxTemp	Rainfall
162	112	0
79	131	29

---

6. Convert Categorical Columns

for col in df.select_dtypes(include='object').columns:

Kya karta hai?

Dataset ke saare text (object type) columns ko find karta hai.

Example:

WindGustDir
RainToday
RainTomorrow

---

df[col] = LabelEncoder().fit_transform(df[col])

Text ko numeric values me convert karta hai.

Example:

RainToday
Yes
No

Ban jayega:

RainToday
1
0

---

7. Features Select Karna

X = df.iloc[:,0:2]

Important

`iloc[:,0:2]`

Matlab:

Saari rows
Column 0 aur Column 1

Tumhare dataset me:

Column Number	Column Name
0	MinTemp
1	MaxTemp

To:

X

contain karega:

MinTemp
MaxTemp

sirf.

---

8. KMeans Model Banana

```
kmeans = KMeans(
    n_clusters=3,
    random_state=42
)
```

n_clusters=3

Data ko:

Cluster 0
Cluster 1
Cluster 2

3 groups me divide karega.

---

random_state=42

Har baar same result mile.

---

9. Fit + Predict

```
df["Cluster"] = kmeans.fit_predict(X)
```

Ye line do kaam karti hai.

Fit

kmeans.fit(X)

Model train karta hai.

Predict

kmeans.predict(X)

Har row ko cluster assign karta hai.

Example:

MinTemp	MaxTemp	Cluster
162	112	0
79	131	2
76	107	2

---

10. Scatter Plot

```
plt.scatter(
    X.iloc[:,0],
    X.iloc[:,1],
    c=df["Cluster"]
)
```

Graph banata hai.

X-axis

X.iloc[:,0]

Means:

MinTemp

Y-axis

X.iloc[:,1]

Means:

MaxTemp

Color

c=df["Cluster"]

Cluster ke according color.

Example:

Cluster 0 → Blue
Cluster 1 → Green
Cluster 2 → Red

---

11. Cluster Centers Plot

```
plt.scatter(
   kmeans.cluster_centers_[:,0],
   kmeans.cluster_centers_[:,1],
```

Cluster centers (centroids) plot karta hai.

Centroid matlab:

Cluster ka average center point.

---

marker='X'

Center ko X symbol se dikhata hai.

---

s=200

X ka size bada rakhta hai.

---

12. Graph Title

plt.title("K-Means Clustering")

Graph ka title.

---

13. Graph Show

plt.show()

Graph display karta hai.

Viva Explanation

Agar examiner pooche:

"Is program me kya kiya?"

Answer:

> "Maine weather dataset load kiya. Categorical columns ko LabelEncoder ki madad se numeric values me convert kiya. MinTemp aur MaxTemp columns ko clustering ke liye select kiya. K-Means algorithm me 3 clusters use kiye. Har record ko ek cluster assign kiya aur scatter plot ki madad se clusters aur unke centroids visualize kiye."

Viva Question

Q: K-Means me `n_clusters=3` ka kya matlab hai?

Answer:

Dataset ko 3 groups (clusters) me divide karna.

Q: `fit_predict()` kya karta hai?

Answer:

Model ko train bhi karta hai aur cluster labels bhi generate karta hai.

Q: `cluster_centers_` kya hota hai?

Answer:

Har cluster ka centroid ya average center point.

SHORT EXPLANATION ABOUT SHORT CODE

Haan, **ye K-Means Clustering ka correct aur simple program hai**. ✅

Bas ek baat dhyan rakho:

X = df.iloc[:,0:2]

Iska matlab sirf **pehle 2 columns** clustering ke liye use honge.

Tumhare weather dataset me pehle 2 columns hain:

MinTemp
MaxTemp

To clustering sirf **MinTemp aur MaxTemp** ke basis par hogi.

Code ka Flow

Dataset Load
df = pd.read_csv("weather.csv")

Dataset load karta hai.

Categorical Columns Encode
for col in df.select_dtypes(include='object').columns:

```
df[col] = LabelEncoder().fit_transform(df[col])
```

Text columns ko numbers me convert karta hai.

Example:

Yes → 1
No → 0

Features Select
X = df.iloc[:,0:2]

Pehle 2 columns:

MinTemp
MaxTemp

select hote hain.

K-Means Model
kmeans = KMeans(n_clusters=3, random_state=42)

3 clusters banenge.

Training + Prediction
df["Cluster"] = kmeans.fit_predict(X)

- Model train hota hai.
- Har row ko cluster number milta hai.

Example:

0
1
2
0
1

Plot Clusters
plt.scatter(X.iloc[:,0], X.iloc[:,1], c=df["Cluster"])

Graph me:

- X-axis = MinTemp
- Y-axis = MaxTemp
- Color = Cluster

Cluster Centers

```
plt.scatter(
  kmeans.cluster_centers_[:,0],
  kmeans.cluster_centers_[:,1],
  marker='X',
  s=200
)
```

Centroids (cluster centers) ko bade **X** mark se show karta hai.

Title

```
plt.title("K-Means Clustering")
```

Graph title.

Show

```
plt.show()
```

Graph display.

BADA WALA CODE KHUD KA

Bilkul. Chalo **K-Means Clustering** ka code line-by-line samajhte hain.

---

1. Pandas Import

```
import pandas as pd
```

- Pandas library import ki.
- CSV file read karne aur data handle karne ke liye.

---

2. Matplotlib Import

import matplotlib.pyplot as plt

- Graph plot karne ke liye.

---

3. Seaborn Import

import seaborn as sns

- Attractive graphs aur scatter plots banane ke liye.

---

4. LabelEncoder Import

from sklearn.preprocessing import LabelEncoder

- Text values ko numbers me convert karne ke liye.

Example:

Wind Direction
N
S
E

Convert:

Wind Direction
0
1
2

---

5. KMeans Import

from sklearn.cluster import KMeans

- K-Means Clustering algorithm import kiya.

---

6. Silhouette Score Import

from sklearn.metrics import silhouette_score

- Clustering kitni achhi hui hai ye measure karne ke liye.

---

7. Dataset Load Karna

df = pd.read_csv("weather.csv")

Weather dataset ko DataFrame me load kiya.

---

8. Encode All Columns

for col in df.columns:

- Dataset ke har column par loop chalega.

---

df[col] = LabelEncoder().fit_transform(df[col].astype(str))

Kya ho raha hai?

Step 1
df[col].astype(str)

Column ko string me convert karta hai.

Step 2
LabelEncoder()

Encoder object create karta hai.

Step 3
fit_transform()

Text values ko numbers me convert karta hai.

Example:

RainToday
Yes
No

Convert:

RainToday
1
0

---

9. Features Select Karna
X = df

Poora dataset features ke roop me use kiya.

K-Means me target variable nahi hota.

---

10. KMeans Model Banana

```
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)
```

n_clusters=3

Data ko 3 groups me divide karega.

---

random_state=42

Har run par same result milega.

---

n_init=10

Algorithm 10 baar run hoga aur best clustering choose karega.

---

11. Model Train Karna

```
kmeans.fit(X)
```

K-Means data par clusters banata hai.

Algorithm:

1. Random centroids choose karta hai.
2. Nearest centroid ke basis par points assign karta hai.
3. Naye centroids calculate karta hai.
4. Repeat karta hai jab tak clusters stable na ho jaye.

---

12. Cluster Labels Add Karna

```
df["Cluster"] = kmeans.labels_
```

Naya column create hua:

Cluster

Example:

Record	Cluster
Row 1	0
Row 2	2
Row 3	1

---

13. Dataset Print

print("Clustered Dataset:")

Heading print karta hai.

---

print(df.head())

Pehli 5 rows display karta hai.

Tumhare output me:

```
RainTomorrow  Cluster
1             0
1             2
1             2
1             2
0             0
```

Matlab har row kisi cluster me assign hui.

---

14. Cluster Centers Print

print("\nCluster Centers:")

Heading print karta hai.

---

print(kmeans.cluster_centers_)

Har cluster ka center print karta hai.

Tumhare output:

```
[[128.68 ...]
 [69.13  ...]
 [82.90  ...]]
```

Meaning

Cluster 0 ka average point.

Cluster 1 ka average point.

Cluster 2 ka average point.

Ye centroids ke coordinates hain.

---

15. Silhouette Score

score = silhouette_score(X, kmeans.labels_)

Clustering quality calculate karta hai.

---

print("\nSilhouette Score:", score)

Tumhara output:

Silhouette Score: 0.26760249404413033

Interpretation

Score	Meaning
Near 1	Excellent
0.5+	Good
0.2 - 0.5	Average
Near 0	Weak

Tumhara:

0.2676

Average clustering hai.

---

16. Figure Size

plt.figure(figsize=(8,6))

Graph size set karta hai.

Width = 8

Height = 6

---

17. Scatter Plot

sns.scatterplot(

Scatter plot banata hai.

---

x=df["WindGustDir"]

X-axis par:

WindGustDir

---

y=df["WindDir9am"]

Y-axis par:

WindDir9am

---

hue=df["Cluster"]

Clusters ke hisab se different colors.

Example:

Cluster 0 → Blue
Cluster 1 → Orange
Cluster 2 → Green

---

s=100

Dot size set karta hai.

---

18. Graph Title

plt.title("K-Means Clustering")

Graph ka title.

---

19. X-axis Label

plt.xlabel("WindGustDir")

X-axis ka naam.

---

20. Y-axis Label

plt.ylabel("WindDir9am")

Y-axis ka naam.

---

21. Graph Show

plt.show()

Final graph display karta hai.

Viva Explanation (Short)

> "Maine weather.csv dataset load kiya aur LabelEncoder ki help se sabhi categorical columns ko numeric values me convert kiya. Poore dataset ko K-Means clustering ke liye use kiya. K=3 choose karke model train kiya aur har record ko ek cluster assign kiya. Cluster centers print kiye aur Silhouette Score calculate kiya, jo clustering quality ko measure karta hai. Scatter plot ki madad se clusters ko visualize kiya."